

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

1. Database Design

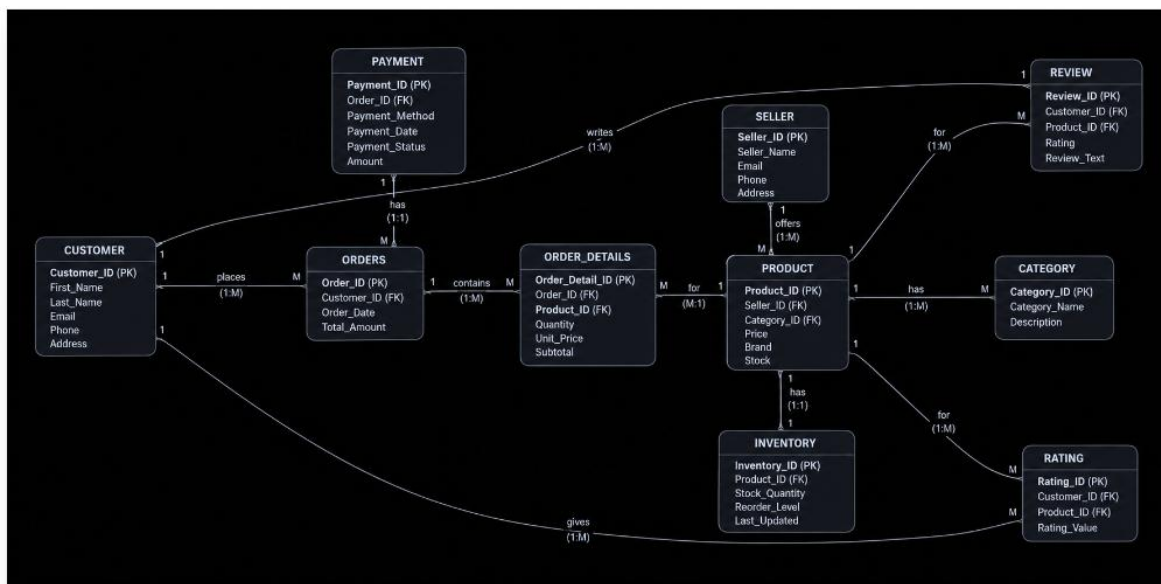
1.1 Review Table

Attribute	Data Type	Key Type / Constraint
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Rating	NUMBER	NOT NULL
Review_Text	VARCHAR2(500)	—

1.2 Rating Table

Attribute	Data Type	Key Type / Constraint
Rating_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Rating_Value	NUMBER	NOT NULL

2. Table Relationships



3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review (  
  2     Review_ID INT PRIMARY KEY,  
  3     Customer_ID INT,  
  4     Product_ID INT,  
  5     Rating INT NOT NULL,  
  6     Review_Text VARCHAR2(500),  
  7     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
  8     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
  9 );
```

Table created.

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
  2     Rating_ID INT PRIMARY KEY,  
  3     Customer_ID INT,  
  4     Product_ID INT,  
  5     Rating_Value INT NOT NULL,  
  6     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
  7     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
  8 );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO Review  
  2 VALUES (801, 201, 101, 5, 'Excellent product');
```

1 row created.

```
SQL>  
SQL> INSERT INTO Review  
  2 VALUES (802, 202, 102, 4, 'Good product');
```

1 row created.

```
SQL>  
SQL> INSERT INTO Review  
  2 VALUES (803, 203, 103, 5, 'Very good quality');
```

1 row created.

5. Rating Data Insertion

```
SQL> INSERT INTO Rating  
  2  VALUES (901, 201, 101, 5);
```

```
1 row created.
```

```
SQL>
```

```
SQL> INSERT INTO Rating  
  2  VALUES (902, 202, 102, 4);
```

```
1 row created.
```

```
SQL>
```

```
SQL> INSERT INTO Rating  
  2  VALUES (903, 203, 103, 5);
```

```
1 row created.
```

6. Retrieve Product Review Details

```
SQL> SELECT
  2     R.Review_ID,
  3     R.Product_ID,
  4     P.Product_Name,
  5     R.Customer_ID,
  6     R.Rating,
  7     R.Review_Text
  8 FROM Review R
  9 JOIN Product P
 10     ON R.Product_ID = P.Product_ID
 11 ORDER BY R.Review_ID;
```

```
REVIEW_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
CUSTOMER_ID  RATING
-----
REVIEW_TEXT
-----
      801      101
iPhone 15
      201      5
Excellent product
```

```
REVIEW_ID PRODUCT_ID
-----
PRODUCT_NAME
```

7. Calculate Average Product Ratings

This uses the AVG() aggregate function.

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Review
  5 GROUP BY Product_ID
  6 ORDER BY Product_ID;
```

```
PRODUCT_ID AVERAGE_RATING
-----
      101          5
      102          4
      103          5
```

8. Identify Highly Rated Products

Products with an average rating of 4 or above can be displayed using HAVING.

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Review
  5 GROUP BY Product_ID
  6 HAVING AVG(Rating) >= 4
  7 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	AVERAGE_RATING
101	5
103	5
102	4

9. Product Rating Report

A combined report can be generated using the Rating and Product tables.

```
SQL> SELECT
  2     R.Rating_ID,
  3     R.Product_ID,
  4     P.Product_Name,
  5     P.Brand,
  6     R.Customer_ID,
  7     R.Rating_Value
  8 FROM Rating R
  9 JOIN Product P
 10     ON R.Product_ID = P.Product_ID
 11 ORDER BY R.Rating_ID;
```

RATING_ID	PRODUCT_ID
PRODUCT_NAME	
BRAND	
CUSTOMER_ID	RATING_VALUE
901	101
iPhone 15	
Apple	
201	5

RATING_ID	PRODUCT_ID
PRODUCT_NAME	